

SOME OBSTRUCTION TO PRIMITIVE CLOSURE

Milan Rosko

May 2026

ORCID: 0009-0003-1363-7158

Abstract

We prove, and mechanize in ROCQ, an abstract obstruction theorem for primitive closure predicates, defined as $C : \mathbf{Form} \rightarrow \mathbf{Prop}$ over the closed implication-falsity fragment $A, B ::= \perp \mid A \rightarrow B$. Two structurally distinct completeness principles for C enter the result. Evaluation completeness $\mathbf{Eval}(C)$ is generative: every formula-valued behavior of codes admits a representing code, up to closure equivalence $A \simeq_C B \triangleq C(A \rightarrow B) \wedge C(B \rightarrow A)$. Excluded-middle completeness $\mathbf{LEM}(C)$ is decisional: every formula is accepted, or its object-level negation is accepted. Yet their conjunction is obstructive: $\mathbf{Eval}(C)$ generates a reflective fixed-point $B \simeq_C \neg B$, which $\mathbf{LEM}(C)$ forces C to classify. Either branch collapses to $C(\perp)$ under modus ponens, and consistency converts the internal collapse into an external contradiction. A **BOOLEAN** decision implies $\mathbf{LEM}(C)$ and is therefore obstructed in turn, whereas refutation imposes no coverage requirement and is inhabited by the always-false classifier.

1 INTRODUCTION

1.1 BACKGROUND

Limitative results are typically established within their native semantics: **INCOMPLETENESS** [Whitehead and Russell, 1927; Gödel, 1931; Rosser, 1936] relies on the arithmetization of syntax; **UNDEFINABILITY** [Tarski, 1933] proceeds by diagonalizing against a hypothesized truth predicate; **SEMIDECIDABILITY** [Church, 1936; Turing, 1937; Kleene, 1952; Rice, 1953] presupposes a fixed model of computation; **LÖB’S THEOREM** [Löb, 1955] depends on the modal derivability conditions. The present paper isolates the diagonal obstruction shared by these results under a minimal set of structural assumptions. Working in a constructive metatheory [Troelstra and van Dalen, 1988] makes the role of bivalence visible: imposing the **LAW OF THE EXCLUDED MIDDLE (LEM)** on the closure predicate is the obstruction’s trigger.

1.2 INTUITION

We convey through a *thought experiment*, prior to any precise definition. Consider a

$$\text{regulator } C \tag{1}$$

whose sole function is to record acceptance for formulas built from implication and falsity. Three properties are at stake. *Detachment*: whenever C accepts both $A \rightarrow B$ and A , it must accept B . *Consistency*: C does not accept $\perp$. And the load-bearing one, *excluded-middle completeness*: every formula A is accepted, or its negation $\neg A$ is accepted. Detachment and consistency are the working hypotheses one would grant any reasonable closure predicate; the third is the totality assumption that the rest of the paper traces back to critique of unrestricted bivalence by Brouwer [1923].

C is to name its evaluative tableau. If every formula-valued behavior of codes admits a representing code, C can be probed against the behavior

“Negate what the input would produce if applied to itself.”

Self-applying that code produces a formula B that, inside C , is identified with its own negation, thence it is a fixed-point of negation at the level of acceptance, not of syntax.

The *informal* reading is simple. To certify the question

“Does LEM hold for every formula?”

one must already answer in LEM’s own form: either it holds, or it does not. *Impredicativity* [Feferman, 1964] sits not in the answer but in the procedure that produces one; the classifier deploys the very pattern whose unrestricted availability it is being asked to certify.

“Pushing” the metaphor one step further: pressed for a verdict on every formula, the regulator would quietly like one of them left off the list. The certificate it would prefer to sign reads

“Every question checks out, except this one, maybe?”

But $\text{LEM}(C)$ is precisely the refusal of that exemption: *every* formula, with no clause excusing even one. The diagonal B is a formula like any other, so the regulator is bound to decide it; and the only formula it cannot afford to decide is exactly B . The sentence the regulator must utter to be complete and the sentence it cannot utter without collapsing are the same sentence. There is no “except this one” to retreat to: that retreat is a fourth principle, weaker than $\text{LEM}(C)$, and the obstruction is the proof that nothing weaker will do and nothing as strong survives.

GÖDEL’S SECOND THEOREM [Gödel, 1931] and the FEFERMAN reflection principles [Feferman, 1962] locate the obstruction one level up: a consistent system that internalizes a survey of its own derivations cannot also certify that survey, because the survey is itself one of the derivations it ranges over. $\text{Eval}(C)$ is that internalization at the propositional core, the regulator naming its own evaluative behaviors, and it leaves no metalevel to retreat to: a verdict withheld is still a verdict the survey records, so declining B is a way of deciding B . In this layer the three principles divide cleanly: $\text{Eval}(C)$ performs the internalization, $\text{LEM}(C)$ demands the certificate, and $\text{Cons}(C)$ is what makes the certificate unaffordable.

“If I can’t answer the question, I answered it.”

The formal fixed-point makes that circularity mechanical. $\text{Eval}(C)$ supplies a formula B with $B \simeq_C \neg B$; $\text{LEM}(C)$ then forces C to commit to B or to $\neg B$. Either commitment rides the accepted equivalence, and detachment alone propagates the chosen branch to $C(\perp)$, which $\text{Cons}(C)$ rules out. Predicativity [Russell, 1908] is therefore the lever: a consistent, detachment-closed regulator that names its own evaluative behaviors under closure cannot also use excluded-middle completeness to certify that very closure.

1.3 OVERVIEW

We define

$$A \simeq_C B \triangleq C(A \rightarrow B) \wedge C(B \rightarrow A). \quad (2)$$

Two local structural properties of C enter the obstruction. The first is closure modus ponens $\text{MP}(C)$, or detachment. The second is closure consistency

$$\text{Cons}(C) \triangleq C(\perp) \rightarrow \perp, \quad (3)$$

the only bridge from internal acceptance of $\perp$ to external contradiction.

The *first* completeness principle is generative. An evaluation frame is complete when every formula-valued behavior of codes has a representing code up to closure equivalence:

$$\text{Eval}(C) \triangleq \forall f. \exists c. \forall x. \text{eval}(c, x) \simeq_C f(x). \quad (4)$$

The codes form an arbitrary type rather than $\mathbb{N}$; the right-hand side is closure equivalence rather than syntactic equality. Representational completeness here means naming behaviors, not deciding formulas. A single application of $\text{Eval}(C)$ to the behavior

$$f := x \mapsto \neg \text{eval}(x, x), \quad (5)$$

evaluated at its own code, produces a formula B with $B \simeq_C \neg B$: a negation fixed-point inside C . The diagonal step is CURRY-style self-application [Curry and Feys, 1958; Sørensen and Urzyczyn, 2006]; in the full-frame presentation, $\text{Eval}(C)$ also has the weak point-surjectivity shape familiar from LAWVERE’S FIXED-POINT THEOREM [Lawvere, 1969], with closure equivalence replacing equality.

The *second* completeness principle is decisional. Excluded-middle completeness asserts global bivalence for the regulator:

$$\text{LEM}(C) \triangleq \forall A \in \text{Form}. C(A) \vee C(\neg A). \quad (6)$$

$\text{Eval}(C)$ closes the regulator under reflection; $\text{LEM}(C)$ demands a global classification; their interaction under diagonalization is the obstruction:

$$\text{Eval}(C) \wedge \text{MP}(C) \wedge \text{Cons}(C) \wedge \text{LEM}(C) \implies \perp. \quad (7)$$

The boundary is sharp at $\text{LEM}(C)$: a BOOLEAN decision $\text{Dec}(C)$ implies $\text{LEM}(C)$ and is obstructed along with it, whereas refutation imposes no coverage requirement and survives. The obstruction is hence excluded-middle completeness itself, not negative information as such.

This is worth contrasting with classical CURRY’S PARADOX, where a syntactic fixed-point $B \leftrightarrow (B \rightarrow \perp)$ yields $\perp$ by assuming B and using it twice inside one assumption block, a use of *contraction* that substructural logics block [Ono, 2003; Restall, 2000]. Section 3 discharges no assumption: every modus-ponens premise is a top-level accepted fact, and $\text{LEM}(C)$ supplies one disjunct of $C(B) \vee C(\neg B)$ as such a fact, freely reusable. Reuse is not eliminated but relocated: an accepted fact of C is an ordinary metatheoretic proposition, and the metatheory contracts hypotheses freely. What is avoided is contraction *within the object logic*, so C need not be contraction-closed.

1.4 ROADMAP

Section 2 fixes the substrate: the closed $\perp / \rightarrow$ syntax, the primitive regulator $C : \text{Form} \rightarrow \text{Prop}$, closure equivalence, the three structural properties $\text{MP}(C)$,

$\text{Cons}(C)$, $\text{LEM}(C)$, the decision/refutation split, and the evaluation frame with its completeness principle $\text{Eval}(C)$. Section 3 builds the negation fixed-point $B \simeq_C \neg B$ from $\text{Eval}(C)$, collapses either $\text{LEM}(C)$ -branch to $C(\perp)$ by detachment, and uses $\text{Cons}(C)$ to reach the contradiction; it then derives the decision obstruction and the refutation non-collapse. Section 4 reports the mechanization and the public L001 contract. Section 5 separates $\text{LEM}(C)$ from metatheoretic excluded middle and situates the result.

2 SUBSTRATE

2.1 OBJECT SYNTAX

Definition 2.1.1 (Regulator theory). A *regulator theory* is the syntactic data that generates a concrete primitive regulator. It is a pair

$$R \triangleq (\pi, T), \quad (8)$$

where π is a regulator logic profile and T is an axiom set of object formulas. A context Γ supplies local assumptions but is not part of the theory itself. Together, R and Γ induce the closure predicate

$$C_{R,\Gamma}(A) \triangleq R; \Gamma \vdash A, \quad (9)$$

where $\vdash$ is checked derivability of A from Γ in R . Hence a “regulator theory” serves as a checked syntactic source of accepted formulas, rather than as a model, truth predicate, or semantic environment.

Definition 2.1.2 (Syntax). The object language is a minimal propositional language, containing only falsity and implication.

$$A, B ::= \perp \mid A \rightarrow B, \quad \neg A \triangleq A \rightarrow \perp. \quad (10)$$

There are no atoms, no propositional variables, no quantifiers, no modalities, and no substitution. We write **Form** for the type of formulas; formulas are finite trees over $\{\perp, \rightarrow\}$.

Lemma 2.1.3 (No formula is its own negation). For our regulator

$$\forall A \in \text{Form}, \quad A \neq \neg A \text{ holds.} \quad (11)$$

Proof. $\neg A = A \rightarrow \perp$ has A as a proper subterm. A finite tree cannot be a proper subterm of itself. ■

Remark 2.1.4. Lemma 2.1.3 forces every fixed-point claim of this paper to live *inside* a closure predicate, never as a syntactic identity of formulas.

2.2 PRIMITIVE REGULATION WITH CLOSURE

Definition 2.2.1 (Primitive regulator with closure predicate). A *primitive regulator* on the formula language is a predicate

$$C : \text{Form} \rightarrow \text{Prop}. \quad (12)$$

We write $C \triangleright A$, or equivalently $C(A)$, read “ C regulates A ” or “ A is accepted by C .” Primitive regulation is the bare acceptance layer: no semantic truth,

model satisfaction, arithmetic provability, or computation model is built in, and $C(A)$ is treated as a mere proposition.

Remark 2.2.2. We also call C a *closure predicate* when emphasizing closure properties. Every closure-level result below is parametric in C and never inspects its definition; regulator-level results follow by specialization.

Definition 2.2.3 (Closure equivalence).

$$A \simeq_C B \triangleq C(A \rightarrow B) \wedge C(B \rightarrow A). \quad (13)$$

Remark 2.2.4. $\simeq_C$ records what C itself accepts about the implication arrows between A and B . By Lemma 2.1.3, $A = \neg A$ never holds, whereas $A \simeq_C \neg A$ may, depending on C .

Definition 2.2.5 (Modus ponens, consistency, excluded-middle completeness). The three structural properties of C are

$$\text{MP}(C) \triangleq \forall A \in \text{Form}. \forall B \in \text{Form}. C(A \rightarrow B) \rightarrow C(A) \rightarrow C(B), \quad (14)$$

$$\text{Cons}(C) \triangleq C(\perp) \rightarrow \perp, \quad (15)$$

$$\text{LEM}(C) \triangleq \forall A \in \text{Form}. C(A) \vee C(\neg A). \quad (16)$$

Remark 2.2.6. $\text{MP}(C)$ is the only object-level inference rule the obstruction uses: detachment alone, no deduction, cut, or further schema. $\text{Cons}(C)$ is the external guard against $C(\perp)$, and $\text{LEM}(C)$ is excluded-middle completeness: C accepts every formula or its negation.

Definition 2.2.7 (Decision and refutation). A *decision* for C is a BOOLEAN classifier whose two branches are sound:

$$\text{Dec}(C) \triangleq \exists d : \text{Form} \rightarrow \{\text{tt}, \text{ff}\}. \quad \begin{array}{l} \forall A \in \text{Form}. d(A) = \text{tt} \rightarrow C(A), \\ \forall A \in \text{Form}. d(A) = \text{ff} \rightarrow C(\neg A). \end{array} \quad (17)$$

A *refutation* is the negative-only counterpart, with no constraint on the false branch:

$$\text{Ref}(C) \triangleq \exists r : \text{Form} \rightarrow \{\text{tt}, \text{ff}\}. \quad \forall A \in \text{Form}. r(A) = \text{tt} \rightarrow C(\neg A). \quad (18)$$

Remark 2.2.8. BOOLEAN totality of d delivers $\text{LEM}(C)$ pointwise, so $\text{Dec}(C) \implies \text{LEM}(C)$, the only implication used below. The converse and the weakness of $\text{Ref}(C)$ are definitional observations, not formal separations. $\text{Ref}(C)$ is realizable by the always-false classifier; its certified non-obstruction endpoint (Theorem 3.3.5) is stated at the regulator level.

2.3 THE EVALUATION FRAME

Definition 2.3.1 (Evaluation frame). Fix an arbitrary type `Code` of *codes*. An *evaluation frame* for C is a record with two fields: an evaluation operation

$$\text{eval} : \text{Code} \rightarrow \text{Code} \rightarrow \text{Form}, \quad (c, x) \mapsto \text{eval}(c, x), \quad (19)$$

and a completeness field, stated up to the relation $\simeq_C$,

$$\text{Eval}(C) \triangleq \forall f : \text{Code} \rightarrow \text{Form}. \exists c. \forall x. \text{eval}(c, x) \simeq_C f(x). \quad (20)$$

The two fields are not independent hypotheses: $\text{Eval}(C)$ quantifies over $\text{eval}(\cdot, \cdot)$, so the frame is a single bundled datum, and supplying a frame means supplying both at once.

Remark 2.3.2. The definition mirrors the artifact: a frame is the ROCQ record `ClosureEvaluationFrame`, with fields `ceval_apply` for (19) and `cevaluation_complete` for (20); the theorems of Section 3 take it as a structure and project the needed field, so $\text{Eval}(C)$ is never a free-standing axiom. $\text{Eval}(C)$ places no definability or computability condition on f , and `Code` is arbitrary. This generosity sharpens the result rather than weakening it, since an obstruction theorem is informative only when each hypothesis is satisfiable alone. $\text{Eval}(C)$ is a witnessing schema, categorically point-surjectivity with equality weakened to closure equivalence, not a recursion-theoretic assumption.

Remark 2.3.3 (Why closure equivalence, and why a frame is not vacuous). The relaxation from $=$ to $\simeq_C$ in (20) is forced. With $=$, the formula-output frame is *provably uninhabited*: its diagonal would yield a syntactic fixed point $\text{eval}(c, c) = \neg(\text{eval}(c, c))$, excluded by Lemma 2.1.3; the artifact records this as `no_formula_evaluation_frame_lemma`, of type $\forall E, \perp$ (and `no_bool_evaluation_frame_lemma` for Boolean output). That proof does not transfer to $\text{Eval}(C)$: it needs $\neg$ fixed-point-free for $=$, whereas $B \simeq_C \neg B$ is acceptance by C , not a tree identity. So $\text{Eval}(C)$ is the equality relaxed exactly far enough to escape the proved impossibility while remaining consistently assumable: an interface, like a group structure, not a claim about a fixed computational model. The mechanization accordingly takes a frame as interface data, never asserting one by a global inhabitation theorem.

Remark 2.3.4 (Goal-restricted frame). The diagonal step of Section 3 does not consume the full field (20). It needs only the *goal-restricted* frame at G , which names the single behavior $x \mapsto \text{eval}(x, x) \rightarrow G$:

$$\text{Eval}_G(C) \triangleq \exists c. \forall x. \text{eval}(c, x) \simeq_C \text{eval}(x, x) \rightarrow G. \quad (21)$$

A full frame yields Eval_G for every G by instantiating $\text{Eval}(C)$ at $f := x \mapsto \text{eval}(x, x) \rightarrow G$; the mechanization records this as `ClosureEvaluationFrameForGoal`.

3 OBSTRUCTION

3.1 FIXED-POINT

Definition 3.1.1 (CURRY'S FIXED-POINT inside C). For a goal formula G , a formula B is a CURRY'S FIXED-POINT for C at G when

$$B \simeq_C B \rightarrow G. \quad (22)$$

Lemma 3.1.2 (CURRY'S FIXED-POINT). Under $\text{Eval}(C)$, for every goal formula G there exists $B \in \text{Form}$ with $B \simeq_C B \rightarrow G$.

Proof. Apply $\text{Eval}(C)$ to $f := x \mapsto \text{eval}(x, x) \rightarrow G$ to obtain a code c with $\text{eval}(c, x) \simeq_C \text{eval}(x, x) \rightarrow G$ for all x . Set $B \triangleq \text{eval}(c, c)$ and specialize at $x := c$. ■

Remark 3.1.3. The proof uses $\text{Eval}(C)$ only at the single behavior $x \mapsto \text{eval}(x, x) \rightarrow G$, that is, only the goal-restricted frame $\text{Eval}_G(C)$ of Remark 2.3.4. The mechanization takes exactly this weaker premise ($\text{ClosureEvaluationFrameForGoal}$); the full frame is invoked here only for notational uniformity.

Remark 3.1.4. Lemma 3.1.2 is the closure-level form of the CURRY’S FIXED-POINT [Curry and Feys, 1958; Sørensen and Urzyczyn, 2006]; its categorical shape matches LAWVERE’S FIXED-POINT THEOREM [Lawvere, 1969] with point-surjectivity replaced by $\text{Eval}(C)$ and equality replaced by closure equivalence. No cartesian-closed categorical structure is assumed in the present substrate.

Lemma 3.1.5 (Generic closure fixed-point). Under $\text{Eval}(C)$, for every transformer $g : \text{Form} \rightarrow \text{Form}$ there exists $B \in \text{Form}$ with $B \simeq_C g(B)$.

Proof. Apply $\text{Eval}(C)$ to $f := x \mapsto g(\text{eval}(x, x))$ to obtain a code c with $\text{eval}(c, x) \simeq_C g(\text{eval}(x, x))$ for all x . Set $B \triangleq \text{eval}(c, c)$ and specialize at $x := c$. ■

Remark 3.1.6. This full-frame variant records the broader transformer fixed-point available from $\text{Eval}(C)$. The obstruction below uses only the Curry instance at $G := \perp$.

Definition 3.1.7 (Negation fixed-point inside C). A formula $B \in \text{Form}$ is a *negation fixed-point* for C when

$$\text{NegFP}_C(B) \triangleq B \simeq_C \neg B. \quad (23)$$

Theorem 3.1.8 (Constructive negation fixed-point). Under $\text{Eval}(C)$, $\exists B \in \text{Form}. \text{NegFP}_C(B)$.

Proof. Apply Lemma 3.1.2 with $G := \perp$. Since $\neg B \triangleq B \rightarrow \perp$, the result is $B \simeq_C \neg B$. Equivalently, apply Lemma 3.1.5 to the transformer $g := A \mapsto \neg A$; the two routes coincide because $\text{NegFP}_C(B)$ and the Curry fixed-point at $G := \perp$ are the same proposition after unfolding $\neg B$. The mechanization takes the transformer route. ■

Remark 3.1.9. This construction supplies the fixed-point fact used throughout the rest of the paper. The proof is a single application of the fixed-point lemma followed by the unfolding $\neg B \triangleq B \rightarrow \perp$. No nonconstructive metatheoretic principle is used, and $\text{LEM}(C)$ does not appear in the proof; $\text{MP}(C)$ and $\text{Cons}(C)$ have not yet been used either. By Lemma 2.1.3, no formula equals its own negation as a syntax tree; in particular the produced B does not; the equivalence is acceptance of the unfolded implications $B \rightarrow (B \rightarrow \perp)$ and $(B \rightarrow \perp) \rightarrow B$ by C itself.

3.2 COLLAPSE AND OBSTRUCTION

Lemma 3.2.1 (Branch collapse). Assume $\text{MP}(C)$ and $\text{NegFP}_C(B)$. Then both

$$C(B) \rightarrow C(\perp), \quad C(\neg B) \rightarrow C(\perp). \quad (24)$$

Proof. Left branch. Assume $C(B)$. From $\text{NegFP}_C(B)$, unfolding $\neg B$, we have $C(B \rightarrow (B \rightarrow \perp))$. $\text{MP}(C)$ yields $C(B \rightarrow \perp)$; a second $\text{MP}(C)$ on this and $C(B)$ gives $C(\perp)$. *Right branch.* Assume $C(\neg B)$, that is, $C(B \rightarrow \perp)$. From $\text{NegFP}_C(B)$, unfolding the reverse direction, we have $C((B \rightarrow \perp) \rightarrow B)$, so $\text{MP}(C)$ gives $C(B)$. Detaching $C(B \rightarrow \perp)$ against $C(B)$ gives $C(\perp)$. ■

Theorem 3.2.2 (Closure-level diagonal collapse). Assume $\text{Eval}(C)$, $\text{MP}(C)$, $\text{LEM}(C)$. Then there exists $B \in \text{Form}$ with

$$\text{NegFP}_C(B) \wedge (C(B) \vee C(\neg B)) \wedge C(\perp). \quad (25)$$

Proof. By Theorem 3.1.8 (using $\text{Eval}(C)$) take B with $\text{NegFP}_C(B)$. $\text{LEM}(C)$ at $A := B$ supplies the disjunction $C(B) \vee C(\neg B)$. Either disjunct gives $C(\perp)$ by Lemma 3.2.1; the three conjuncts of (25) are then $\text{NegFP}_C(B)$, the $\text{LEM}(C)$ disjunction itself, and that $C(\perp)$. ■

Remark 3.2.3. The middle conjunct of (25) is retained because the mechanized endpoint bundles all three as a single existential, recording the branch taken; Theorem 3.2.4 consumes only the third.

Theorem 3.2.4 (APORETIC LEMMA).

$$\text{Eval}(C) \wedge \text{MP}(C) \wedge \text{Cons}(C) \wedge \text{LEM}(C) \implies \perp. \quad (26)$$

Proof. Theorem 3.2.2 yields $C(\perp)$, $\text{Cons}(C)$ converts it to $\perp$. ■

Remark 3.2.5. The four hypotheses enter at four distinct steps: $\text{Eval}(C)$ produces B once (Theorem 3.1.8), $\text{LEM}(C)$ is instantiated once at B (Theorem 3.2.2), $\text{MP}(C)$ fires twice on the chosen branch (Lemma 3.2.1), and $\text{Cons}(C)$ converts $C(\perp)$ into $\perp$ once (Theorem 3.2.4).

3.3 DECISION AND REFUTATION

Lemma 3.3.1 (Decision collapse). Assume $\text{Eval}(C)$, $\text{MP}(C)$, $\text{Dec}(C)$. Then there exists $B \in \text{Form}$ with

$$\text{NegFP}_C(B) \wedge (C(B) \vee C(\neg B)) \wedge C(\perp). \quad (27)$$

Proof. $\text{Dec}(C) \implies \text{LEM}(C)$ by case analysis on $d(B)$ (Definition 2.2.7); the conclusion is then (25) of Theorem 3.2.2. No consistency hypothesis enters. ■

Theorem 3.3.2 (Decision obstruction).

$$\text{Eval}(C) \wedge \text{MP}(C) \wedge \text{Cons}(C) \wedge \text{Dec}(C) \implies \perp. \quad (28)$$

Proof. Lemma 3.3.1 yields $C(\perp)$; $\text{Cons}(C)$ converts it to $\perp$. Equivalently, $\text{Dec}(C) \implies \text{LEM}(C)$ and Theorem 3.2.4 applies directly. ■

Theorem 3.3.3 (Ad absurdum impossibility). Assume $\text{MP}(C)$ and $\text{Cons}(C)$. If $\text{NegFP}_C(B)$ holds for some B and $\text{Dec}(C)$ holds, then $\perp$.

Proof. $\text{Dec}(C) \implies \text{LEM}(C)$; instantiate at B , apply Lemma 3.2.1, then $\text{Cons}(C)$. ■

Remark 3.3.4. Theorem 3.3.3 is the hypothesis-weakened core of Theorem 3.3.2: the latter spends $\text{Eval}(C)$ only to manufacture B (Theorem 3.1.8), whereas the former drops $\text{Eval}(C)$ and assumes $\text{NegFP}_C(B)$ directly. The diagonal is thus not the load-bearing step of the *decision* obstruction; any negation fixed-point suffices, and both endpoints are bundled in L001 .

Theorem 3.3.5 (Refutation is not obstructed). There exists $r : \text{Form} \rightarrow \{\text{tt}, \text{ff}\}$ such that

$$\forall A \in \text{Form}. r(A) = \text{ff}, \quad (29)$$

and the branch soundness condition $r(A) = \text{tt} \rightarrow C(\neg A)$ holds vacuously. Hence $\text{Ref}(C)$ is inhabited without assuming $\text{Cons}(C)$, $\text{MP}(C)$, or $\text{Eval}(C)$.

Proof. Set $r(A) := \text{ff}$ for every A . The soundness condition has an empty antecedent, so it is satisfied without invoking either consistency or the diagonal formula B . ■

Remark 3.3.6. The mechanized endpoint `refutation_not_obstructed` carries $\text{Cons}(C)$ as a hypothesis of its signature, but the proof discharges it unused: the witness r and its soundness field are independent of consistency, of $\text{MP}(C)$, and of $\text{Eval}(C)$. The hypothesis is retained in the type only to state the result uniformly with the obstruction endpoints; it is not used. The statement is regulator-level; no generic closure-level refutation theorem is formalized, since at the closure level the claim is the immediate vacuous one above.

Remark 3.3.7. Refutation imposes no global coverage requirement, whereas a decision must classify every formula into a positive or negative accepted branch. Combining Theorems 3.2.4, 3.3.2, and 3.3.5, the proved contrast is

$$\begin{aligned} (\text{Eval}(C), \text{MP}(C), \text{Cons}(C), \text{Dec}(C)) &\implies \perp, \\ (\text{Eval}(C), \text{MP}(C), \text{Cons}(C), \text{LEM}(C)) &\implies \perp, \end{aligned} \quad (30)$$

$\text{Ref}(C)$ is inhabited unconditionally.

The boundary is exactly excluded-middle completeness: $\text{Dec}(C)$ and $\text{LEM}(C)$ commit C on every formula and collapse, whereas $\text{Ref}(C)$ makes no such commitment and survives.

4 MECHANIZATION

Mechanization 4.0.1. The statements above are formalized in `GALLINA`, the specification language of:



The substrate of Section 2 is `M001`: the closed $\perp / \rightarrow$ syntax, a `BOOLEAN` proof checker, deduction and reductio transformers, syntactic adequacy of checked derivability, and the evaluation-frame interface. The evaluation frame is exposed as explicit interface data, not asserted by a global inhabitation theorem.

[Github.com](#) $\rightarrow$ [Milan-Rosko](#) $\rightarrow$ [Proofcase](#) $\rightarrow$ `M001`

The obstruction of Section 3 is L001, layered above M001. Its headline theorems are the negation fixed point (Theorem 3.1.8), the aporetic obstruction (Theorem 3.2.4), and the regulator specializations:

```
negfixp_existence
aporetic_obstruction
regulator_aporetic_obstruction
regulator_decision_obstruction
```

The refutation non-obstruction (Theorem 3.3.5) is formalized only at the regulator level, with no generic closure-level refutation type, since its content is the vacuous soundness of the always-false classifier. The certified endpoint is:

```
refutation_not_obstructed
witnessed by always_false_regulator_refutation
```

The assumption report is taken at the terminal endpoints `certified_aporetic_lemma_contract` and `aporetic_lemma_qed`, whose `Print Assumptions` redirects are empty; every theorem above is a dependency of that contract and therefore inherits the same empty profile: no axioms, no admits.

[Github.com](#) → [Milan-Rosko](#) → [Proofcase](#) → L001

The public L001 API bundles seven endpoints, in the fixed order of the conjunction `APORETIC_LEMMA_CONTRACT`. Three conclude an existence or an internal acceptance: `APORETIC_FIXED_POINT_CONTRACT` (Theorem 3.1.8), `APORETIC_EXCLUDED_MIDDLE_COLLAPSE_CONTRACT` (Theorem 3.2.2), and `APORETIC_DECISION_COLLAPSE_CONTRACT` (Lemma 3.3.1), the decision-hypothesis counterpart of the second, both stopping at internal $C(\perp)$ without the consistency conversion. The other four conclude the external $\perp$: `APORETIC_EXCLUDED_MIDDLE_OBSTRUCTION_CONTRACT` (Theorem 3.2.4), `APORETIC_DECISION_OBSTRUCTION_CONTRACT` (Theorem 3.3.2, which spends $\text{Eval}(C)$ to build the fixed point), `APORETIC_EQUIVALENCE_AD_ABSURDUM_IMPOSSIBILITY_CONTRACT` (Theorem 3.3.3, which drops $\text{Eval}(C)$ and instead assumes $\text{NegFP}_C(B)$), and `APORETIC_DECIDER_IMPOSSIBILITY_CONTRACT`, the same impossibility for a decider supplied as an existential. The terminal certificate

$$\text{aporetic_lemma_qed} : \text{APORETIC_LEMMA_CONTRACT} \quad (31)$$

inhabits that bundled type and is exported through the artifact layer alongside `Print Assumptions` redirects. The construction does not import classical logic, choice, propositional or functional extensionality, or any modal or arithmetic axiom. No OCAML extraction is performed: the obstruction endpoints live in `Prop`, whose proof content is erased under extraction, so the meaningful artifact is the assumption report rather than executable code; the computational layer is the `BOOLEAN` proof checker of M001.

5 DISCUSSION

5.1 MECHANISM

The central observation is that representational completeness generates a reflective domain. Formula-valued behaviors of codes are named by codes

themselves up to closure equivalence; self-application of such a code produces a formula with closure-level fixed-point status, without syntactic self-reference or arithmetic encoding.

The obstruction arises at the interaction between two structures. $\text{Eval}(C)$ establishes reflection closure; $\text{LEM}(C)$ imposes a binary verdict on every formula. Because the diagonal B is built from C 's own evaluative content, any $\text{LEM}(C)$ -verdict on B propagates through the fixed-point arrows by modus ponens, yielding $C(\perp)$, which $\text{Cons}(C)$ rules out.

The upshot is therefore neither an inconsistency in excluded middle nor a defect in evaluation completeness, taken in isolation. Rather, bivalence is incompatible with reflection in the presence of detachment and consistency, once evaluation is internalized up to closure equivalence. The roles of the assumptions separate sharply: $\text{Eval}(C)$ is generative and structural; $\text{LEM}(C)$ is global and classificatory; $\text{MP}(C)$ is local compositionality; $\text{Cons}(C)$ enforces external soundness. Only their interaction under reflection produces the obstruction.

5.2 WITNESSING SCHEMATA

Read as witness patterns, $\text{Eval}(C)$ and $\text{LEM}(C)$ share a common $\forall. \exists$. profile. $\text{Eval}(C)$ is

$$\forall f : \text{Code} \rightarrow \text{Form}. \exists c. \forall x. \text{eval}(c, x) \simeq_C f(x), \quad (32)$$

which produces, for every formula-valued behavior, a code witnessing it up to closure equivalence. $\text{LEM}(C)$ (Definition 2.2.5) is the disjunction

$$\forall A \in \text{Form}. C(A) \vee C(\neg A); \quad (33)$$

rewritten existentially, this is

$$\forall A \in \text{Form}. \exists \varepsilon \in \{A, \neg A\}. C(\varepsilon). \quad (34)$$

Both schemata then have the same $\forall. \exists$. profile: $\forall \cdot \exists \cdot$. (closure-relation). They differ in the size of the witness set per index: $\text{Eval}(C)$ draws witnesses from the full code type Code , while $\text{LEM}(C)$ draws from the binary set $\{A, \neg A\}$. Among binary-witness schemas of this profile indexed over Form , $\text{LEM}(C)$ is the minimal informal choice that commits C on every formula.

Read this way, $\text{Eval}(C)$ and $\text{LEM}(C)$ are not unrelated assumptions but two instances of the same totality discipline applied along complementary axes. $\text{Eval}(C)$ is reflective totality on the *generation* side: every behavior is named. $\text{LEM}(C)$ is reflective totality on the *classification* side: every formula is committed. $\text{LEM}(C)$ ranges over all formulas; $\text{Eval}(C)$ ranges over all formula-valued behaviors, including the diagonal behavior used to construct B . Neither schema exempts the diagonal. The pairing analyzed here imposes both totalities at once, and the diagonal collapse of Theorem 3.2.4 is its formal cost.

This places the obstruction within a wider family. A trivial-witness variant,

$$\forall A. \exists \varepsilon \in \{A\}. C(\varepsilon), \quad (35)$$

is inconsistent already at $A := \perp$ under $\text{Cons}(C)$. The empty-witness variant, $\forall A. \exists \varepsilon \in \emptyset. C(\varepsilon)$, is unsatisfiable. $\text{LEM}(C)$ sits between these as the binary-

witness instance that imposes a real classification while remaining satisfiable in isolation, and is the principle whose interaction with $\text{Eval}(C)$ we analyzed.

5.3 REFUTATION ASYMMETRY

$\text{Ref}(C)$, as a *conditional* principle rather than a totality, sits outside the witnessing family of Section 5.2. Nothing forces r to fire on the diagonal B , and the always-false classifier shows that consistency is compatible with the principle inhabited vacuously. The asymmetry is therefore not about negative information as such, but about the absence of a uniform existential commitment indexed by every formula. What the obstruction rules out is the totality discipline applied to classification; what it leaves untouched is the implicational discipline of refutation.

5.4 CLASSICAL COMPARISONS

The result can be read as a generalization of classical diagonal phenomena, stripped of their specific encoding mechanisms. The present closure layer is purely propositional: its only primitives are a closure predicate, an evaluation frame, and detachment. The common core is representability-induced reflection together with global bivalence under detachment; consistency then converts the resulting internal collapse into contradiction. Compared with a purely categorical presentation, the formulation here is operational: the interface is expressed through codes, evaluation, and a closure predicate. Rather than a literal subsumption of the classical theorems, Theorem 3.2.4 obstructs $\text{LEM}(C)$ as a hypothesis on the primitive regulator, not classical excluded middle as a metatheoretic axiom.

5.5 OUTLOOK

The regulator-theory framing isolates a transferable acceptance interface, and the natural next step is to ask whether the obstruction transfers along it. A reduction from one regulator to another that preserves $\text{MP}(C)$, $\text{Cons}(C)$, and the evaluation frame would carry the obstruction with it, so that obstructedness becomes a property closed under such reductions rather than a fact about one fragment. Mapping which reductions preserve each hypothesis, and which break $\text{LEM}(C)$ without breaking the rest, would chart the boundary as a structure rather than a single point.

REFERENCES AND NOTES

REFERENCES

- L. E. J. Brouwer. Über die Bedeutung des Satzes vom ausgeschlossenen Dritten in der Mathematik, insbesondere in der Funktionentheorie. *Journal für die reine und angewandte Mathematik*, 154:1–7, 1923. URL <https://doi.org/10.1515/crll.1925.154.1>.
- A. Church. A note on the Entscheidungsproblem. *Journal of Symbolic Logic*, 1(1):40–41, 1936. URL <https://doi.org/10.2307/2269326>.
- H. B. Curry and R. Feys. *Combinatory Logic, Volume I*. North-Holland, Amsterdam, 1958. ISBN 9780720422078.
- S. Feferman. Transfinite recursive progressions of axiomatic theories. *Journal of Symbolic Logic*, 27(3):259–316, 1962. URL <https://doi.org/10.2307/2964649>.
- S. Feferman. Systems of predicative analysis. *Journal of Symbolic Logic*, 29(1):1–30, 1964. URL <https://doi.org/10.2307/2269764>.
- K. Gödel. Über formal unentscheidbare Sätze der Principia Mathematica und verwandter Systeme I. *Monatshefte für Mathematik und Physik*, 38(1):173–198, 1931. URL <https://doi.org/10.1007/BF01700692>.
- S. C. Kleene. *Introduction to Metamathematics*. North-Holland, 1952. ISBN 9780444896230.
- F. W. Lawvere. Diagonal arguments and Cartesian closed categories. In *Category Theory, Homology Theory and their Applications, II*, volume 92 of *Lecture Notes in Mathematics*, pages 134–145. Springer, 1969. URL <https://doi.org/10.1007/bfb0080769>.
- M. H. Löb. Solution of a problem of Leon Henkin. *Journal of Symbolic Logic*, 20(2):115–118, 1955. URL <https://doi.org/10.2307/2266895>.
- H. Ono. Substructural Logics and Residuated Lattices — an Introduction. In *Trends in Logic: 50 Years of Studia Logica*, Trends in Logic, pages 193–228. Springer Netherlands, Dordrecht, 2003. URL https://doi.org/10.1007/978-94-017-3598-8_8.
- G. Restall. *An Introduction to Substructural Logics*. Routledge, London, 2000. URL <https://doi.org/10.4324/9780203016244>.
- H. G. Rice. Classes of Recursively Enumerable Sets and Their Decision Problems. *Transactions of the American Mathematical Society*, 74(2):358–366, 1953. URL <https://doi.org/10.2307/1990888>.
- J. B. Rosser. Extensions of Some Theorems of Gödel and Church. *Journal of Symbolic Logic*, 1(3):87–91, 1936. URL <https://doi.org/10.2307/2269028>.
- B. Russell. Mathematical logic as based on the theory of types. *Proceedings of the Cambridge Philosophical Society*, 30(3):222–262, 1908. URL <https://doi.org/10.2307/2369948>.
- M. H. Sørensen and P. Urzyczyn. *Lectures on the Curry–Howard Isomorphism*, volume 149 of *Studies in Logic and the Foundations of Mathematics*. Elsevier, 2006. URL [https://doi.org/10.1016/s0049-237x\(06\)x8001-1](https://doi.org/10.1016/s0049-237x(06)x8001-1).
- A. Tarski. *Pojęcie prawdy w językach nauk dedukcyjnych*. Number 34 in *Prace Towarzystwa Naukowego Warszawskiego, Wydział III – Nauk matematyczno-fizycznych*. Nakładem Towarzystwa Naukowego Warszawskiego, Warszawa, 1933.
- A. S. Troelstra and D. van Dalen. *Constructivism in Mathematics*, volume 121 of *Studies in Logic and the Foundations of Mathematics*. Elsevier, 1988. URL [https://doi.org/10.1016/s0049-237x\(09\)x7019-9](https://doi.org/10.1016/s0049-237x(09)x7019-9).
- A. Turing. On Computable Numbers, with an Application to the Entscheidungsproblem. *Proceedings of the London Mathematical Society*, s2-42(1):230–265, 01 1937. ISSN 0024-6115. URL <https://doi.org/10.1112/plms/s2-42.1.230>.
- A. N. Whitehead and B. Russell. *Principia Mathematica*. Cambridge University Press, Cambridge, 2nd edition, 1927. ISBN 0521626064.

FINAL REMARKS

The author welcomes criticism, proposed extensions, scholarly correspondence and constructive dialogue. No conflicts of interest are declared. This research received no funding.

Milan Rosko

ORCID: 0009-0003-1363-7158

Email: Q1012878@studium.fernuni-hagen.de

Email: hi-at-milanrosko.com

Licensed under “Deed” ©

creativecommons.org/licenses/by/4.0